

SmileCare Dental Clinic Management System

Project Document

Project: Full-Stack Dental Clinic Management System
Frontend: React, Vite, React Router
Backend: Node.js, Express REST API
Database: PostgreSQL
Local API: `http://localhost:4000`
Local Frontend: `http://localhost:5173`

1. Project Overview

SmileCare is a responsive clinic management system for managing doctors and patient appointments. Clinic staff can view, create, edit, search, filter, and delete records through a web interface. All CRUD actions are routed through the backend API and are PostgreSQL-compatible.

The application includes a dashboard with clinic summaries, a doctors management page, and an appointments management page.

2. Objectives

- Build a modern component-based React application.
- Implement client-side navigation with React Router.
- Build a reliable Express REST API.
- Persist doctors and appointments in a relational database.
- Validate data on both frontend and backend.
- Provide useful loading, empty, success, and error states.
- Create a deployment-ready project structure.

3. Application Pages

Dashboard

- Total doctors count
- Upcoming appointments count
- Today's appointments summary
- Upcoming appointments list
- Doctors list
- Appointments list
- Appointment details
- Doctor details
- Appointment form
- Doctor form
- Appointment filters
- Doctor filters
- Appointment search
- Doctor search
- Appointment status
- Doctor status
- Appointment history
- Doctor history
- Appointment analytics
- Doctor analytics
- Appointment reports
- Doctor reports
- Appointment settings
- Doctor settings
- Appointment notifications
- Doctor notifications
- Appointment logs
- Doctor logs
- Appointment audit
- Doctor audit
- Appointment backup
- Doctor backup
- Appointment restore
- Doctor restore
- Appointment export
- Doctor export
- Appointment import
- Doctor import
- Appointment delete
- Doctor delete
- Appointment update
- Doctor update
- Appointment create
- Doctor create

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

ÄiÄfÆ'Äçä,ÄiÄfäesÄ,ÄiÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Äçä,ÄÄÄfÄçÄçäesÄÄÄçä€ÄçÄfÆ'Ätä€™ÄfäesÄ,ÄçÄfÆ'Ä,ÄçÄfÄçÄçäesÄÄÄfäesÄ,ÄÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄiÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Ä,ÄçÄfÄçÄçä,ÄÄiÄ,ÄÄÄfä€Ä,ÄiÄfÆ'Ätä€™ÄfÄçÄçäesÄÄÄ... ÄiÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄiÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Äçä,ÄÄÄfÄçÄçäesÄÄÄçä€ÄçÄfÆ'Ätä€™ÄfÄçÄçäesÄÄÄ,ÄÄfÆ'Ä,ÄçÄj ÄiÄfäesÄ,ÄÄfÆ'Ä,ÄçÄfÄçÄçäesÄÄÄ... ÄÄÄfäesÄ,ÄçÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Äçä,ÄÄÄfÄçÄçäesÄÄÄçä€ÄçÄfÆ'Ätä€™ÄfäesÄ,ÄçÄfÆ'Ä,ÄçÄfÄçÄçäesÄÄÄÄ... ÄiÄfäesÄ,ÄÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄçÄfÆ'Ätä€™ÄfäesÄ,ÄçÄfÆ'Ä,ÄçÄfÄçÄçä,ÄÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄçÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Äçä,ÄÄÄfÄçÄçäesÄÄÄçä€ÄçÄfÆ'Ätä€™ÄfÄçÄçäesÄÄÄ... ÄiÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄçÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Äçä,ÄÄÄfÄçÄçäesÄÄÄçä€ÄçÄfÆ'Ätä€™ÄfÄçÄçäesÄÄÄÄ... ÄiÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄiÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Ä,ÄçÄfÄçÄçä,ÄÄiÄ,ÄÄÄfä€Ä,ÄiÄfÆ'Ätä€™ÄfÄçÄçäesÄÄÄ... ÄiÄfäesÄ,ÄÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄiÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Ä,ÄçÄfÄçÄçä,ÄÄiÄ,ÄÄÄfä€Ä,ÄiÄfÆ'Ätä€™ÄfÄçÄçäesÄÄÄ... ÄiÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄiÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Äçä,ÄÄÄfÄçÄçäesÄÄÄçä€ÄçÄfÆ'Ätä€™ÄfÄçÄçäesÄÄÄ,ÄÄfÆ'Ä,ÄçÄj ÄiÄfäesÄ,ÄÄfÆ'Ä,ÄçÄfÄçÄçäesÄÄÄ... ÄÄÄfäesÄ,ÄçÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Äçä,ÄÄÄfÄçÄçäesÄÄÄçä€ÄçÄfÆ'Ätä€™ÄfÄçÄçäesÄÄÄÄ... ÄiÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Ä,ÄçÄfÄçÄçä,ÄÄiÄ,ÄÄÄfäesÄ,ÄiÄfÆ'Ätä€™ÄfÄçÄçäesÄÄÄÄ... ÄiÄfÆ'Äçä,ÄÄiÄfäesÄ,ÄiÄfÆ'Ätä€™Äfä€ Äçä,Äa,çÄfÆ'Ä,ÄçÄfÄçÄçä,ÄÄiÄ,ÄÄÄfä€Ä,ÄiÄfÆ'Ätä€™ÄfÄçÄçäesÄÄÄÄ... ÄiÄfÆ'Äçä,ÄÄiÄfäesÄ,Äçs appointments count

- Pending appointments count
- Recent appointments list
- Loading and error feedback

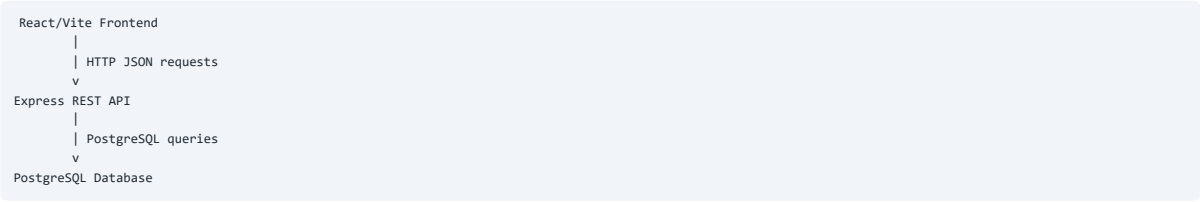
Doctors

- Search doctors by name, specialization, or email
- View doctor cards
- View detailed doctor information
- Add doctor
- Edit doctor
- Delete doctor with confirmation
- Active/inactive status display
- Availability display

Appointments

- Search by patient or reason
- Filter by appointment status
- Add appointment
- Edit appointment
- Delete appointment with confirmation
- Doctor selector
- Date/time field
- Status field
- Loading and empty states

4. Architecture



Frontend

The frontend is located in `client/`. It contains React pages, reusable forms, modal dialogs, navigation, API calls, loading states, and responsive CSS.

Backend

The backend is located in `server/`. It contains Express routes, validation, error handling, seed fallback data, PostgreSQL queries, and the health endpoint.

Database

The database schema is in `server/schema.sql`. The `appointments.doctor_id` column references `doctors.id` with a foreign key.

5. Database Model

Doctors

Column	Type	Description
id	SERIAL	Primary key
name	VARCHAR	Doctor name
specialization	VARCHAR	Medical specialization
phone	VARCHAR	Contact phone
email	VARCHAR	Contact email
availability	TEXT	Working hours
active	BOOLEAN	Active/inactive status

Appointments

Column	Type	Description
id	SERIAL	Primary key
patient_name	VARCHAR	Patient name
patient_contact	VARCHAR	Patient contact
doctor_id	INTEGER	Foreign key to doctors
date_time	TIMESTAMP	Appointment date and time
reason	TEXT	Appointment reason
status	VARCHAR	pending, confirmed, completed, cancelled

6. API Routes

Health

```
GET /api/health
```

Doctors

```
GET    /api/doctors
POST   /api/doctors
PUT    /api/doctors/:id
DELETE /api/doctors/:id
```

Appointments

```
GET    /api/appointments
POST   /api/appointments
PUT    /api/appointments/:id
DELETE /api/appointments/:id
```

Successful responses use:

```
{
  "success": true,
  "data": {}
}
```

Error responses use:

```
{
  "success": false,
  "error": "Useful error message"
}
```

7. Validation and Business Rules

- Required doctor fields are validated on frontend and backend.
- Required appointment fields are validated on frontend and backend.
- Appointment status must be one of `pending`, `confirmed`, `completed`, or `cancelled`.
- Invalid date/time values are rejected.
- Invalid doctor references are rejected.
- A doctor cannot be deleted while related appointments exist in PostgreSQL.
- Overlapping appointments for the same doctor within one hour are rejected.
- Destructive deletes require confirmation.
- Lists refresh after every successful mutation.

8. Local Setup

```
cd E:\PF\chatbot\Task1
npm install
```

Copy `.env.example` to `.env`:

```
PORT=4000
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/dental_clinic
VITE_API_URL=http://localhost:4000/api
```

Run the database schema:

```
server/schema.sql
```

Start the application:

```
npm run dev
```

Open:

```
http://localhost:5173
```

If `DATABASE_URL` is not configured, the application uses local seed data in memory for UI demonstration.

9. Testing Notes

The following checks were completed:

- `npm install` completed successfully.
- `node --check server/index.js` passed.
- Vite production build passed.
- `/api/health` returned `status: ok`.
- Seed doctors endpoint returned data.
- Invalid overlapping appointment returned HTTP `409`.
- Frontend routes were included for dashboard, doctors, and appointments.
- CRUD forms include validation and confirmation behavior.

Major Evidence Screenshot Placement

```
cd E:\PF\chatbot\Task1
>> npm run dev
[0] > dev:api
[0] > node server/index.js
[0]
[1]
[1] > dev:web
[1] > vite --config client/vite.config.js
[1]
[0] ◇ injected env (3) from .env
[0] API running
[1]
[1] VITE v6.4.3 ready in 894 ms
[1]
[1] → Local: http://localhost:5173/
[1] → Network: use --host to expose
[1] 4:04:45 PM [vite] (client) page reload .codex-browser-profile/Default/Extensions/Temp/scoped_dir12564_533497816/CRX_INSTALL/src/popup/index.html
[1] 4:04:46 PM [vite] (client) page reload .codex-browser-profile/Default/Extensions/Temp/scoped_dir12564_533497816/CRX_INSTALL/src/popup/index.html
[1] 4:04:47 PM [vite] (client) page reload .codex-browser-profile/Default/Extensions/aopddeflghjljihihabdclejbojaomaf/1.4.25_0/src/popup/index.html
[1] 4:04:48 PM [vite] (client) page reload .codex-browser-profile/Default/Extensions/nmmhkkegcccagdldgiimedpiccmgmieda/1.0.0.6_0/html/craw_window.html
[1] 4:04:49 PM [vite] (client) page reload .codex-browser-profile/Default/Extensions/ghbmnnjooekpmoecnnnilnnbdlolhkhi/1.110.1_0/offscreendocument.html
[1] 4:09:12 PM [vite] (client) page reload .codex-browser-profile/WasmTtsEngine/20260904.1/offscreen.html
```

 SmileCare

DashboardDoctorsAppointments

SMILECARE CLINIC

Good morning

Total doctors

2

Upcoming appointments

1

Today's appointments

0

Pending status

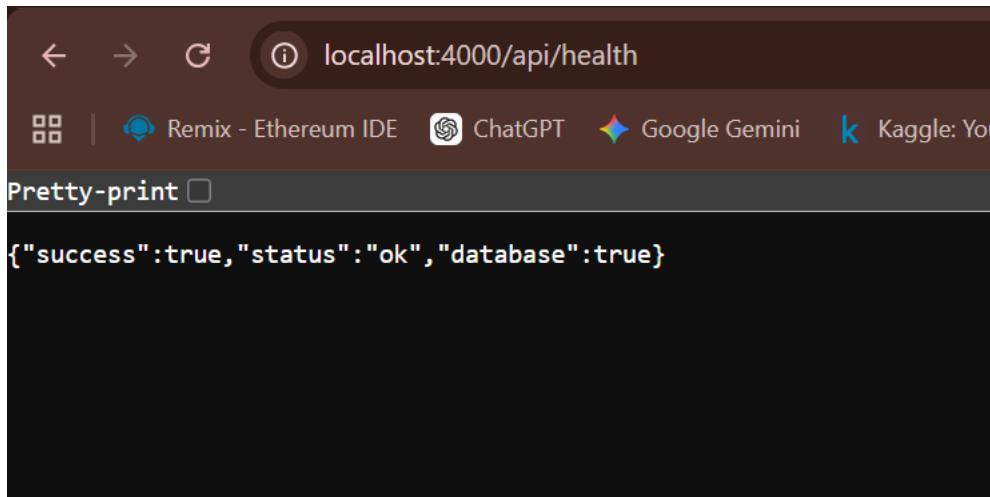
1

Recent appointments

Bilal Raza

Braces consultation

9/23/2026, 2:30:00 PM · pending



10. Deployment Notes

Render was evaluated for deployment, but the available Blueprint flow required a paid plan/payment method. For the working public demonstration, the project was deployed through Vercel.

Public Links

- GitHub develop branch: <https://github.com/AsadAliEng/Dental-Clinic-System>
- Vercel frontend deployment: <https://dental-clinic-system-3f11zd4e4-asad-6cc9.vercel.app/>
- Vercel frontend project URL: <https://dental-clinic-system-chi.vercel.app/>
- Vercel backend API: <https://dental-clinic-api-omega.vercel.app/>
- API health endpoint: <https://dental-clinic-api-omega.vercel.app/api/health> The frontend uses `VITE_API_URL=https://dental-clinic-api-omega.vercel.app/api` and the API is deployed from the `server` directory. Neon/remote PostgreSQL was intentionally skipped because it required a paid setup. Local PostgreSQL was connected, schema-applied, and verified successfully; the public Vercel API uses its documented in-memory fallback.

11. Loom Walkthrough

The walkthrough should demonstrate:

- Dashboard metrics
- Navigation between all pages
- Doctor create, edit, view, search, and delete
- Appointment create, edit, filter, and delete
- Validation error
- Empty/loading/error states
- API health endpoint
- Live deployment

Clickable Loom URL: [Watch the Loom walkthrough](#)

12. Known Limitations

- Authentication and role-based access are not included.
- Remote PostgreSQL was intentionally skipped for the public demo; local PostgreSQL was verified successfully, while the deployed API uses an in-memory fallback.
- Automated browser tests are not included.
- Appointment duration is represented by a one-hour conflict rule.
- Screenshots, public deployment links, Loom link, and Google Drive link must be added before final submission.

13. Final Submission Checklist

- ☒ Public GitHub repository

- ☒ Code pushed to `develop` branch
 - ☒ Frontend Vercel URL
 - ☒ Backend Vercel URL
 - ☒ Working `/api/health` endpoint
 - ☒ Screenshots added
 - ☒ Loom URL added as clickable link
 - ☒ PDF exported
 - ☐ PDF uploaded to Google Drive
 - ☐ Google Drive link made public
-

Submission Link Summary

- GitHub repository (`develop` branch): <https://github.com/AsadAliEng/Dental-Clinic-System>
- Frontend Vercel project: <https://dental-clinic-system-chi.vercel.app/>
- Latest frontend deployment: <https://dental-clinic-system-3f11zd4e4-asad-6cc9.vercel.app/>
- Backend API: <https://dental-clinic-api-omega.vercel.app/>
- API health endpoint: <https://dental-clinic-api-omega.vercel.app/api/health>
- Loom walkthrough: <https://www.loom.com/share/e87f5c848fee4e3b83b1b9ab0831334a>
- Public Google Drive PDF link: **Add after uploading** `AsadAli_Task1_Report.pdf`
- Public Google Drive project document link: **Add after uploading** `AsadAli_Task1_Submission_Details.md`

Render required a paid plan/payment method, so Vercel was used as the deployment alternative. Local PostgreSQL was verified; Neon/remote PostgreSQL was intentionally skipped.